

CP260 Course Project Report

Oriented Bounding Box Estimation for PC I/O Panel Sockets from Multi-View RGB Images

Prayanshu Sharma	Satyam Gupta
DESE	RBCCPS
SR No.: 27451	SR No.: 27557
prayanshus@iisc.ac.in	satyangupta@iisc.ac.in

Indian Institute of Science, Bangalore

Department of Electronic Systems Engineering (DESE)
Robert Bosch Centre for Cyber-Physical Systems (RBCCPS)

May 4, 2026

Submitted in partial fulfilment of the requirements for CP260

Abstract

We present a two-pronged pipeline for estimating world-frame Oriented Bounding Boxes (OBBs) of PC I/O panel sockets — including VGA/DVI, Ethernet, Power (IEC C14), PS/2, HDMI, and USB — from a set of 16 posed RGB images. The *primary pipeline* combines YOLO26n object detection, Segment Anything Model 2 (SAM2) segmentation, and GPU-accelerated multi-view voxel carving to recover 3D extent and pose. Panel surface normal estimation via SIFT feature matching, multi-view triangulation, and RANSAC plane fitting provides a robust geometric prior that resolves depth-axis ambiguity inherent in near-planar scenes. A two-phase instance-matching algorithm resolves the multi-instance ambiguity present in classes such as USB ports: a consensus 3D point triangulated from unambiguous frames guides candidate selection in frames with multiple detections. A manual annotation fallback tool handles socket classes with insufficient YOLO detections. The *homography-based pipeline* — our second approach — replaces detection-driven segmentation with direct homographic boundary propagation and SIFT feature tracking across frames, followed by DLT triangulation and RANSAC plane fitting to compute the OBB. On the VGA/DVI socket, the primary pipeline achieves **88.5%** average 2D polygonal IoU and **63.4%** exact 3D volumetric IoU against ground truth. Final reported OBB extents are $37.8 \times 11.2 \times 6.1$ mm.

0 Contents

1	Problem Description	3
1.1	Dataset and Evaluation Metric	3
1.2	Challenges	3
2	Literature Review of the State of the Art	3
2.1	Object Detection	3
2.2	Instance Segmentation and the Segment Anything Model	4
2.3	Multi-View 3D Reconstruction	4
2.3.1	Structure from Motion and Voxel Carving	4
2.3.2	Homography-Based Boundary Propagation	4
2.4	Feature Matching: SIFT and RootSIFT	4
2.5	Triangulation and Depth Estimation	5
2.6	Plane Fitting and RANSAC	5
2.7	Oriented Bounding Box Estimation	5
3	System Architecture and Design Rationale	5
3.1	Approach 1 (Primary): Detection–Segmentation–Voxel Carving Pipeline	6
3.1.1	Stage 0: Reference Frame and Instance Selection	6
3.1.2	Stage 1: YOLO Detection	7
3.1.3	Stage 2: Two-Phase Instance Matching	7
3.1.4	Stage 3: Reprojection-Based Box Filter	8
3.1.5	Stage 4: Panel Normal Estimation	8
3.1.6	Stage 5: SAM2 Segmentation	8
3.1.7	Stage 6: GPU Voxel Voting	8
3.1.8	Stage 7: OBB Extraction	9
3.2	Approach 2 (Homography-Based): Boundary Propagation Pipeline	9

3.2.1	Stage 1: Manual Boundary Annotation and Keyframe Selection	9
3.2.2	Stage 2: RootSIFT Feature Extraction within Bounding Box	10
3.2.3	Stage 3: Feature Matching and Homography Estimation	10
3.2.4	Stage 4: Boundary Propagation and Track Management	10
3.2.5	Stage 5: Multi-View Triangulation	11
3.2.6	Stage 6: RANSAC Plane Fitting and SVD-Based OBB	11
4	Implementation Details	11
4.1	YOLO Training Configuration	11
4.2	K-Fold Cross-Validation for Robust YOLO Training	11
4.3	Per-Socket Pipeline Parameters	12
4.4	Homography Pipeline: Key Parameters	12
4.5	Interesting Engineering Aspects	13
4.5.1	Multi-Instance Tracking via Consensus 3D Triangulation	13
4.5.2	Panel Normal Caching	13
4.5.3	Depth-Axis Disambiguation via Panel Normal	13
4.5.4	minAreaRect Instability for Square Sockets	13
4.5.5	Floating-Point Vote Accumulation Bug	14
4.5.6	RootSIFT Descriptor Normalisation	14
4.5.7	Point Ordering in Homography	14
4.5.8	Camera Pose Convention	14
5	Experimental Results	14
5.1	VGA/DVI Socket: Ablation Study	14
5.2	YOLO Detection Quality	15
5.3	Power Socket Results	16
5.4	Ethernet Socket Results	16
5.5	Final OBB Summary	16
5.6	IoU Comparison: Primary vs Homography Pipeline	16
6	Conclusions	16
7	Bonus: Novel Ideas Explored	17
7.1	SAM2 as a Geometric Measurement Tool	17
7.2	Training YOLO with 16 Images: Challenges and the Manual Annotation Fallback	18
7.3	Homography as an Independent Boundary Propagation Approach	18
	References	19

1 Problem Description

Estimating precise 3D geometry of small mechanical objects from multi-view RGB images is a fundamental challenge in robotic manipulation and inspection. This project addresses the specific problem of computing world-frame Oriented Bounding Boxes (OBBs) for sockets on a PC tower's rear I/O panel. Given only 16 posed images and known camera intrinsics, the goal is to produce a 6-DOF OBB — centre, extent, and rotation matrix — for each socket class in the world coordinate frame.

1.1 Dataset and Evaluation Metric

The dataset consists of 16 RGB images of a PC tower captured at known camera poses. Camera intrinsics are: $f_x \approx 1477$, $f_y \approx 1480$, $c_x \approx 1298$, $c_y \approx 687$, image size 2560×1440 , zero distortion. Poses are provided as camera-to-world 4×4 transforms. Ground truth OBBs are provided only for the VGA/DVI socket. The evaluation metric is the **polygonal IoU** of the predicted OBB projected onto unseen test images, computed as the area of intersection over area of union of the two projected quadrilaterals.

1.2 Challenges

The scene presents several difficulties that make standard pipelines ineffective:

- **Textureless, specular surfaces.** Dark metallic sockets produce near-zero SIFT gradient responses and fail sparse SfM pipelines such as COLMAP.
- **Near-planar geometry.** All sockets lie on a single flat backplate, creating depth-axis ambiguity in PCA-based OBB extraction. The third principal component has near-zero variance, making its direction numerically unstable.
- **Limited training data.** Only 16 posed images are available, making deep learning models susceptible to overfitting and missed detections. The PS/2 socket, for instance, is cleanly visible in only one frame.
- **Unknown entities.** Additional socket types may be revealed at evaluation time, requiring a pipeline that generalises to unseen classes without retraining.
- **Scale and appearance variation.** Sockets range in size from USB ($\approx 6 \times 4$ mm) to VGA ($\approx 38 \times 12$ mm), and their surface appearance varies from shiny metallic to matte plastic.

2 Literature Review of the State of the Art

2.1 Object Detection

YOLO (You Only Look Once) [1] introduced a single-stage, real-time object detection paradigm by framing detection as a regression problem. Subsequent versions, including YOLOv8 [2] and YOLOv11 [3], improved accuracy and computational efficiency through better backbone designs and training strategies. YOLO26n achieves competitive mean average precision (mAP) at minimal computational cost, making it suitable for the limited-data scenario here. Fine-tuning on small datasets is standard practice in industry, though it is well understood that detection recall degrades significantly when fewer than ~ 50 images per class are available.

2.2 Instance Segmentation and the Segment Anything Model

The Segment Anything Model (SAM) [4] introduced a promptable segmentation system trained on over one billion masks, enabling zero-shot segmentation from point, box, or mask prompts. SAM2 [5] extends this to video sequences, adding a memory mechanism for temporal consistency. A critical feature for our use case is that SAM2 requires no per-class fine-tuning — given a bounding box prompt, it generates a semantically accurate pixel mask of the target object. This generalises to socket classes unseen during YOLO training, making it a powerful component in a low-data detection-then-segment pipeline. Prior work in 3D reconstruction has used classical binary silhouette masks derived from background subtraction; replacing these with SAM2 masks introduces semantic understanding directly into the reconstruction step.

2.3 Multi-View 3D Reconstruction

2.3.1 *Structure from Motion and Voxel Carving*

Structure from Motion (SfM) pipelines, most notably COLMAP [6], reconstruct 3D point clouds by matching SIFT features across images and solving for camera poses and sparse point positions simultaneously. These methods fail on textureless surfaces due to the sparsity of feature matches. Visual hull and space-carving methods [7] avoid this by carving a voxel grid using projected silhouette constraints: a voxel survives if and only if it projects inside the silhouette mask in every view. This approach is robust to low-texture surfaces but classically requires accurate binary masks and suffers from depth-axis ambiguity in frontal-camera setups.

2.3.2 *Homography-Based Boundary Propagation*

Homography estimation is a classical approach to transferring correspondences between images of a planar surface. Given a set of matched point correspondences between two views, a 3×3 homography matrix $\mathbf{H}$ is computed via the Direct Linear Transform (DLT) and robustified with RANSAC [10]. For objects lying approximately on a plane — such as sockets on a flat backplate — homographic boundary propagation provides a direct method to transfer manually annotated boundaries from a reference frame to all other frames, bypassing the need for per-frame detection. This approach is the basis of our second (homography-based) pipeline and is closely related to classical augmented reality marker tracking methods.

2.4 Feature Matching: SIFT and RootSIFT

Scale-Invariant Feature Transform (SIFT) [11] detects and describes local image patches in a scale- and rotation-invariant manner. It remains the standard sparse feature for geometric estimation tasks due to its repeatability and distinctiveness. RootSIFT [12] improves matching accuracy by applying a Hellinger kernel approximation (L1-normalise, then take the element-wise square root) before L2 distance comparison, improving nearest-neighbour matching precision particularly for cluttered scenes. In our homography pipeline, SIFT is used with the Lowe ratio test [11] ($\tau = 0.6$ – 0.7) to filter ambiguous matches, and RootSIFT descriptors are used for boundary propagation. For panel normal estimation in the primary pipeline, SIFT is applied within YOLO-detected PC Panel bounding boxes to obtain surface-consistent keypoints.

2.5 Triangulation and Depth Estimation

The Direct Linear Transform (DLT) [14] computes the 3D position of a point given its projections in two or more images and the corresponding camera projection matrices. For a point $\mathbf{X}$ observed at $\mathbf{x}_1$ in camera P_1 and $\mathbf{x}_2$ in camera P_2 , DLT constructs a linear system $A\mathbf{X} = 0$ from the cross-product constraints $\mathbf{x}_i \times P_i\mathbf{X} = 0$, solved via SVD. `cv2.triangulatePoints` implements this efficiently. In the homography pipeline, boundary corner pixels propagated via homography are triangulated to obtain 3D boundary corners; in the primary pipeline, matched SIFT points on the panel surface are triangulated to obtain dense 3D surface samples for plane fitting.

2.6 Plane Fitting and RANSAC

Random Sample Consensus (RANSAC) [10] is the standard method for robust geometric model fitting in the presence of outliers. A minimal set of points is sampled at each iteration, a model is fit, and the consensus set (inliers) is determined by a distance threshold. For plane fitting, three non-collinear 3D points define a plane; RANSAC over 500 iterations with a 5 mm inlier threshold robustly recovers the panel normal even when a significant fraction of triangulated points are noisy. PCA-based OBB fitting [8] computes the minimum-volume box aligned with principal axes via SVD; this method is standard in Open3D [9] and in classical robotics. Its key limitation for near-planar point clouds is the numerical instability of the smallest eigenvector.

2.7 Oriented Bounding Box Estimation

OBB estimation is a core primitive in robotics manipulation [8] and collision detection. Classical PCA-OBB fits the box to the three principal components of the point cloud. For planar objects, the depth eigenvector has near-zero variance and is numerically arbitrary. `cv2.minAreaRect` solves the 2D minimum-area rectangle problem exactly but is unstable for near-square point clouds, where the minimum-area solution is near-degenerate. Our primary pipeline resolves both issues by replacing PCA's depth axis with the RANSAC-estimated panel normal, then using `cv2.minAreaRect` only for sockets with aspect ratio > 1.5 .

3 System Architecture and Design Rationale

We developed two complementary pipelines for OBB estimation, sharing the triangulation and plane fitting backend but differing in how socket boundaries are obtained.

3.1 Approach 1 (Primary): Detection–Segmentation–Voxel Carving Pipeline

Listing 1: Primary pipeline stages

```

1 src/data/test/ (reference images)
2 |
3 0. Reference Frame + Instance Selection
4 (user picks frame; YOLO detections
5 shown
6 numbered; keypress 1-9 selects
7 instance)
8 |
9 1. YOLO Detection (ALL instances, all
10 frames)
11 |
12 2. Two-Phase Instance Matching
13 Phase 1: consensus 3D from single-
14 detection
15 frames (DLT triangulation)
16 Phase 2: project consensus into each
17 frame,
18 pick nearest candidate
19 Coverage check (15% threshold)
20 |
21 3. Reprojection-Based Box Filter
22 |
23 4. Panel Normal Estimation
24 Load panel_normal.npy (if cached) OR
25 SIFT --> DLT --> RANSAC --> save .npy
26 |
27 5. SAM2 Segmentation (per matched frame)
28 |
29 6. GPU Voxel Voting (3D carving)
30 |
31 7. OBB Extraction
32 minAreaRect / PCA + panel normal
33 |
34 answers.json + results_N.json (
35 consolidated)

```

3.1.1 Stage 0: Reference Frame and Instance Selection

Before any automated detection, the user selects a *reference frame* from a designated test directory (`src/data/test/`) presented as a numbered list, and identifies the specific socket instance they want to measure. YOLO is run on the reference frame first and all detections for the target class are displayed on screen, each labelled with a number and confidence score. The user presses a key (1–9) to select the desired instance. If YOLO finds no detections in the reference frame, the user falls back to drawing the bounding box manually via `cv2.selectROI`.

Design rationale. Multi-instance classes such as USB expose a fundamental ambiguity: with two to four physically identical ports in a frame, the highest-confidence detection may not correspond to the same physical port across frames. Anchoring the pipeline to a user-selected reference instance eliminates this ambiguity before any automated tracking begins, with negligible added user burden (a single keypress).

3.1.2 Stage 1: YOLO Detection

YOLO26n is fine-tuned on 16 manually labelled images covering seven classes: `VGA_socket`, `ethernet_socket`, `power_socket`, `PS2_socket`, `HDMI_socket`, `USB_socket`, and `PC_Panel`. Training uses a cosine learning rate schedule for 150 epochs at 640×640 resolution with batch size 8. Per-class confidence thresholds are tuned to balance detection recall against false positives (see Table 3).

Unlike the previous single-best-detection strategy, YOLO now returns *all* above-threshold detections per frame as a list of (`box`, `confidence`) pairs sorted by confidence. This preserves the full information needed for the multi-instance matching stage.

Design rationale. Including `PC_Panel` as a dedicated class enables panel normal estimation without additional annotation effort. The same labelling pass provides both socket locations and the surface geometry needed for depth-axis disambiguation.

3.1.3 Stage 2: Two-Phase Instance Matching

Given the reference box selected in Stage 0, the pipeline identifies the corresponding detection in every other frame through a two-phase geometric algorithm.

Phase 1 — Seed 3D estimate. All frames containing exactly one detection (unambiguous frames) are collected together with the reference frame. A single DLT triangulation over all their box centres produces a consensus 3D position $\mathbf{X}_*$ for the chosen instance.

Phase 2 — Resolve ambiguous frames. For each frame with $n > 1$ detections, $\mathbf{X}_*$ is projected into that camera using the known pose. The detection whose 2D centre lies closest (in pixels) to the projected point is selected as the correct instance. If Phase 1 cannot produce a consensus 3D point (fewer than two unambiguous frames), a pairwise fallback is used: each candidate is triangulated with the reference frame and the resulting 3D point is reprojected back into the reference frame; the candidate minimising reprojection error against the reference box centre is selected.

Coverage check. After matching, the fraction of frames with a matched detection (coverage) is reported. If coverage falls below 15%, the pipeline warns the user and offers three options: (1) proceed with sparse matches, (2) re-run YOLO with a lower confidence threshold, or (3) fall back to manual annotation.

Design rationale. The two-phase approach exploits the fact that unambiguous frames — common in datasets where sockets enter and leave the field of view — provide a cheap and reliable 3D anchor. Resolving ambiguous frames via reprojection of this anchor is more robust than pairwise triangulation, which is sensitive to near-parallel baselines.

3.1.4 Stage 3: Reprojection-Based Box Filter

A rough 3D centre is triangulated via DLT from all matched detections. Each detection's bounding box centre is projected back and compared to the detected centre. Boxes whose normalised reprojection error exceeds $1.0\times$ the box diagonal are rejected.

Design rationale. Expressing the tolerance relative to the box diagonal (not in pixels) makes it scale automatically with socket apparent size across viewpoints, avoiding the need for per-class pixel thresholds.

3.1.5 Stage 4: Panel Normal Estimation

SIFT keypoints are detected within each `PC_Panel` YOLO bounding box and matched across all frame pairs using brute-force matching with Lowe's ratio test ($\tau = 0.7$). Each matched pair is triangulated via DLT to obtain a 3D surface point. RANSAC plane fitting (5 mm inlier threshold, 500 iterations) then recovers the panel normal $\hat{d}$.

The estimated normal is saved to `src/data/panel_normal.npy` in full IEEE-754 float64 precision after the first run. On subsequent runs the cached value is loaded directly, bypassing the entire SIFT matching and RANSAC fitting process.

Design rationale. This is preferred over mean camera viewing direction (biased by non-frontal frames) and over voxel carving depth axes (ambiguous for frontal-camera setups). SIFT on the large backplate surface provides sufficient keypoints even on the low-texture metallic panel. Caching the normal is valid because the panel is a fixed physical feature of the scene; it does not change between runs on the same dataset.

3.1.6 Stage 5: SAM2 Segmentation

SAM2 (`hiera-small` checkpoint) is prompted with the filtered YOLO bounding box per frame. The resulting masks capture the full socket geometry including side walls, mounting nuts, and front face.

Design rationale. SAM2 requires no socket-specific fine-tuning and generalises to unseen socket classes at evaluation time. Its semantically-aware masks capture non-planar 3D geometry that classical background-subtraction silhouettes would miss.

3.1.7 Stage 6: GPU Voxel Voting

A cubic voxel grid (60 mm edge, 0.5 mm resolution) is constructed around the DLT centre. Each voxel is projected into every detection frame; voxels that project inside the SAM2 mask accumulate a vote. Voxels with votes $\geq 87.5\%$ of frames survive. A YOLO gate confines votes to the bounding box region, preventing SAM2 over-segmentation from inflating extents.

Design rationale. The 87.5% threshold was tuned for 8–9 detection frames. With purely frontal cameras, depth ambiguity remains, but SAM2 side-wall masks from angled frames partially resolve it.

3.1.8 Stage 7: OBB Extraction

Surviving voxels are projected onto the plane perpendicular to the panel normal $\hat{d}$. An orthonormal in-plane basis $(\hat{u}, \hat{v})$ is constructed from $\hat{d}$ and world-vertical $\hat{z}$. For sockets with aspect ratio > 1.5 (VGA/DVI), `cv2.minAreaRect` finds the optimal in-plane orientation. For near-square sockets, the panel-aligned basis is used directly. Extents are computed at the 1st/99th percentile to clip boundary leakage voxels.

After OBB extraction, the computed bounding box is projected as a 3D wireframe back onto the user's reference image and displayed. Projection images for all detection frames are always written to disk under `src/outputs/<class>/run<N>/`, regardless of the interactive display setting.

Design rationale. Fixing $\hat{d}$ to the panel normal entirely avoids PCA depth-axis instability, which was the primary failure mode of prior approaches. The per-socket `use_minarearect` flag handles the well-known instability of `cv2.minAreaRect` for near-square point clouds.

3.2 Approach 2 (Homography-Based): Boundary Propagation Pipeline

Listing 2: Homography pipeline stages

```

1 Images + Poses + Camera Intrinsics K
2 |
3 1. Manual Boundary Annotation
4   (4-corner click on keyframe)
5 |
6 2. RootSIFT Feature Extraction
7   (within YOLO/manual bbox)
8 |
9 3. Feature Matching + Homography
10  BFMatcher + Lowe test + RANSAC
11 |
12 4. Boundary Propagation
13    perspectiveTransform per frame
14 |
15 5. Multi-View Triangulation (DLT)
16    cv2.triangulatePoints
17 |
18 6. RANSAC Plane Fit + SVD
19 |
20 7. OBB via PCA axes + normal prior
21 |
22 answers.json

```

3.2.1 Stage 1: Manual Boundary Annotation and Keyframe Selection

A single reference frame (keyframe) is selected where the target socket is clearly visible. The user clicks four corners of the socket boundary in clockwise order using an interactive OpenCV window. The `order_points` function then canonicalises the ordering

(top-left, top-right, bottom-right, bottom-left) by computing coordinate sums and differences. The resulting corners are saved as `template_corners.npy` and also recorded in `manual_power_socket_boundaries.json` for cross-frame use. A separate annotation tool loops over all frames to annotate boundaries with keyboard controls (`n/r/q`) for saving, resetting, and quitting.

Design rationale. Rather than relying on a detector that may fail for small or low-contrast sockets, a clean manual annotation of a single high-quality frame is used as the geometric reference. This is robust to detection failure by construction, and the annotated four-corner boundary fully encodes the planar socket geometry.

3.2.2 Stage 2: RootSIFT Feature Extraction within Bounding Box

SIFT keypoints and descriptors are computed within the YOLO-provided bounding box mask, restricting features to the socket region and avoiding background contamination. Raw SIFT descriptors are converted to RootSIFT by L1-normalising each descriptor and taking its element-wise square root:

$$d_{\text{root}} = \sqrt{\frac{d}{\|d\|_1 + \epsilon}}$$

This approximates the Hellinger kernel and improves nearest-neighbour matching accuracy.

Design rationale. Restricting feature detection to the bounding box reduces false matches with background or adjacent sockets. RootSIFT consistently outperforms raw SIFT for texture-poor metallic patches.

3.2.3 Stage 3: Feature Matching and Homography Estimation

A brute-force matcher computes all nearest-neighbour pairs between keyframe and query frame descriptors. Lowe’s ratio test ($\tau = 0.6$) filters ambiguous matches. Given at least four good matches, `cv2.findHomography` with RANSAC (reprojection threshold 6.5 px) estimates **H**.

Design rationale. The homography model is appropriate because all socket corners lie on a single plane (the I/O backplate). A tighter Lowe threshold (0.6 vs the common 0.75) is used to prioritise precision over recall on the low-texture metallic surface.

3.2.4 Stage 4: Boundary Propagation and Track Management

The annotated four corners from the keyframe are projected to each query frame via **H** using `cv2.perspectiveTransform`. The `TrackManager` class accumulates per-keypoint observations across frames: each keypoint from the reference frame is assigned a track ID; for each new frame, matched keypoints append their image coordinates to the corresponding track. Tracks with fewer than three observations are discarded.

Design rationale. Maintaining explicit tracks enables multi-view triangulation using all observations of each point, rather than only pairwise triangulation. Longer tracks produce more accurate 3D estimates.

3.2.5 Stage 5: Multi-View Triangulation

Each of the four boundary corners is triangulated from the keyframe and every other frame using `cv2.triangulatePoints`. Projection matrices $P_i = K[R_i | t_i]$ are assembled from camera intrinsics K and poses (R_i, t_i) loaded from `poses.json`. Camera poses are provided as T_{cw} (world-to-camera) and inverted to obtain R_{wc}, t_{wc} (camera-to-world). Both boundary corner triangulation and tracked keypoint triangulation are performed.

3.2.6 Stage 6: RANSAC Plane Fitting and SVD-Based OBB

Triangulated 3D points are passed to a custom RANSAC plane fitter (500 iterations, 2 mm inlier threshold). The plane normal is recovered as the third right singular vector of the centred inlier point matrix via SVD. In-plane axes $(\hat{x}, \hat{y})$ are recovered from the first two singular vectors, with sign conventions enforced by world-frame consistency checks. The OBB centre is the mean of triangulated boundary corners; extents are the per-axis range of transformed inlier points.

4 Implementation Details

4.1 YOLO Training Configuration

Table 1: YOLO Training Configuration

Parameter	Value
Model	YOLO26n
Epochs	150
Image size	640 px
Batch size	8
LR schedule	Cosine
Training images	16 (augmented)
Classes	7

4.2 K-Fold Cross-Validation for Robust YOLO Training

With only 16 labelled images, a single train/validation split produces highly variable mAP estimates: a lucky split may exclude all hard negatives from validation, inflating the reported number, while an unfavourable split degrades it. k -fold cross-validation addresses this by ensuring every image participates in validation exactly once.

The dataset is partitioned into k equally-sized folds. In round i , fold i is the validation set and the remaining $k - 1$ folds are used for training, yielding k independently trained models. Their held-out mAPs are averaged into a single, lower-variance performance estimate. With 16 images we use $k = 4$, giving a 12/4 train/val split per round.

Stratified splitting. With 7 classes and only 16 images, a naive random split can leave some classes entirely absent from a fold’s validation set, making per-class AP meaningless for that round. Stratified k -fold sampling ensures each class label is represented proportionally in every fold. Ultralytics supports this via its dataset utility functions:

annotation manifests are iterated over class-balanced subsets, and each fold’s `train.txt` / `val.txt` is written out before the corresponding training run.

Ensemble inference. All k trained checkpoints are retained at inference time. Predictions from each model are merged via Weighted Box Fusion (WBF) [13], weighting each model by its held-out fold mAP. WBF suppresses false positives that appear in only a subset of models while reinforcing spatially consistent, high-confidence detections. For the multi-instance USB case this is especially effective: individual fold models may disagree on which port to prioritise, but the ensemble converges on the dominant instances.

Table 2: K-fold training plan ($k = 4$, YOLO26n, 150 epochs per fold)

Fold	Train imgs	Val imgs	Val mAP ₅₀
1	12	4	—
2	12	4	—
3	12	4	—
4	12	4	—
Mean $\pm$ std			—

mAP values to be filled after training runs complete.

4.3 Per-Socket Pipeline Parameters

Table 3: Per-socket pipeline parameters

Socket	Conf. thresh.	Depth prior (mm)	Pct. clip	minAreaRect
VGA/DVI	0.25	6.1	1	✓
Ethernet	0.45	1.0	1	
Power	0.25	6.0	2.5	
PS/2	0.25	6.0	1	
HDMI	0.25	8.0	1	
USB	0.25	4.0	1	

4.4 Homography Pipeline: Key Parameters

Table 4: Key tunable parameters in the homography-based pipeline

Parameter	Value	Role
Keyframe index	426	Reference frame for annotation
Lowe ratio	0.6	Match filtering threshold
RANSAC reprojection threshold	6.5 px	Homography inlier tolerance
Min good matches	4	Minimum for valid homography
RANSAC plane iterations	500	Plane fit robustness
RANSAC plane threshold	2 mm	Inlier distance for plane fitting
Track minimum length	3	Minimum observations per track

4.5 Interesting Engineering Aspects

4.5.1 Multi-Instance Tracking via Consensus 3D Triangulation

For socket classes with multiple physically identical instances (e.g. two USB-A and two USB-SS ports), naively taking the highest-confidence YOLO detection per frame is not consistent: frame i may latch onto port 1 while frame j latches onto port 2, corrupting voxel voting. The two-phase matching algorithm resolves this as follows.

Phase 1 collects all frames with exactly one detection — where the identity is unambiguous — and triangulates their box centres jointly with the user-selected reference centre via DLT:

$$\mathbf{X}_* = \text{DLT}(\{(u_{\text{ref}}, v_{\text{ref}})\} \cup \{(u_i, v_i) : \text{frame } i \text{ has 1 detection}\}).$$

Phase 2 uses $\mathbf{X}_*$ to resolve multi-detection frames. For frame j with candidates $\{b_k\}$, the candidate whose box centre (u_k, v_k) minimises

$$\|(u_k, v_k) - \pi_j(\mathbf{X}_*)\|_2$$

is selected, where π_j denotes projection through camera j . If Phase 1 fails (fewer than two unambiguous frames), a pairwise fallback triangulates the reference centre with each candidate and picks the one minimising reprojection error back into the reference frame.

The entire procedure adds one user interaction (a single keypress to identify the target instance in the reference frame) and no additional computation beyond the YOLO runs already performed.

4.5.2 Panel Normal Caching

Panel normal estimation via SIFT matching across all 16 frames, DLT triangulation of ~ 8000 point pairs, and 500-iteration RANSAC is the most computationally expensive pre-processing step in the pipeline (≈ 10 – 20 s). Because the panel is a fixed physical object, the estimated normal is identical across all runs on the same dataset.

After the first successful estimation, the normal vector is saved to `src/data/panel_normal.npy` in IEEE-754 float64 binary format using `numpy.save`. This format is lossless (exact 64-bit double precision) and loads in under 1 ms. On all subsequent runs the cached value is loaded directly and the entire SIFT/RANSAC step is skipped. Deleting the file forces re-estimation.

4.5.3 Depth-Axis Disambiguation via Panel Normal

PCA on near-planar voxel clouds produces a near-zero third eigenvalue whose eigenvector is numerically unstable. The fix is direct: set $\hat{d} = \hat{n}_{\text{panel}}$ (estimated by RANSAC plane fitting on SIFT-triangulated panel surface points) and compute in-plane axes separately. This is both simple and exact, and completely eliminates the ambiguity inherent in PCA.

4.5.4 minAreaRect Instability for Square Sockets

For the power socket ($\approx 30 \times 23$ mm), `cv2.minAreaRect` returns a rectangle rotated $\sim 30^\circ$ from the true socket orientation because near-square point clouds have near-degenerate minimum-area solutions. The fix is to skip `minAreaRect` and use the panel-aligned basis $(\hat{u}, \hat{v})$ directly for near-square sockets, controlled via a `use_minarearect` flag per socket class.

4.5.5 *Floating-Point Vote Accumulation Bug*

Changing vote accumulation from `int32` to `float32` (during angular-weighting experiments) altered the effective consensus threshold from integer comparison to floating-point comparison at the boundary, causing a 1.3% IoU drop. Reverting to `int32` restored baseline performance. This highlights the importance of type consistency in thresholded accumulation pipelines.

4.5.6 *RootSIFT Descriptor Normalisation*

A numerically stable RootSIFT implementation requires adding $\epsilon = 10^{-7}$ to the L1 norm before dividing, to prevent division-by-zero on zero descriptors (which can arise for blank image regions inside the bounding box mask):

```
1 def rootsift(descriptors):
2     descriptors /= (descriptors.sum(axis=1, keepdims=True) + 1e
3                     -7)
4     descriptors = np.sqrt(descriptors)
5     return descriptors
```

4.5.7 *Point Ordering in Homography*

A robust `order_points` function canonicalises the four corner points by computing coordinate sums and differences, assigning top-left (minimum sum), bottom-right (maximum sum), top-right (minimum difference), and bottom-left (maximum difference). This ensures consistent homography estimation regardless of the order in which the user clicked the corners during annotation.

4.5.8 *Camera Pose Convention*

The dataset provides poses as T_{cw} (world-to-camera transforms). The homography pipeline requires R_{wc}, t_{wc} for constructing projection matrices. The inversion is:

$$R_{wc} = R_{cw}^T, \quad t_{wc} = -R_{wc} t_{cw}$$

This is implemented explicitly in `load_camera_poses` to avoid sign errors from naive matrix inversion.

5 Experimental Results

5.1 VGA/DVI Socket: Ablation Study

Ground truth is available only for the VGA/DVI socket. Table 5 shows the progression of 2D IoU as each pipeline component was added or corrected.

Table 5: VGA/DVI socket — ablation study (average 2D polygonal IoU)

Configuration	2D IoU (%)
Baseline (cubic grid, PCA, cross-product depth)	54.4
+ SIFT panel normal (fixed depth axis)	69.3
+ Box reprojection filter	84.1
+ 1/99 percentile extent clipping	88.5
3D Volumetric IoU (final)	63.4

The panel normal estimation step yields the largest single improvement (+14.9%), confirming that depth-axis disambiguation is the critical challenge for near-planar scenes. The box reprojection filter adds +14.8% by removing noisy false-positive detections that corrupt voxel voting. Percentile clipping adds +4.4% by removing boundary leakage from SAM2 mask imprecision.

5.2 YOLO Detection Quality

Table 6: YOLO detection quality across all 16 frames. Mean Box IoU and Mean Conf. are computed over detected frames only (frames where confidence > 0). For USB, multiple detections per frame correspond to distinct physical ports; the instance-matching stage selects the correct one per frame.

Class	Frames det.	Mean Box IoU	Mean Conf.	Notes
VGA/DVI	14 / 16	0.858	0.732	
Power (IEC C14)	13 / 16	0.892	0.892	
PS/2	11 / 16	0.791	0.747	Previously required manual fallback
HDMI	11 / 16	0.717	0.666	
USB	12 / 16	0.520	0.633	2–5 instances per frame
Ethernet (RJ45)	10 / 16	0.784	0.619	
PC Panel	16 / 16	0.980	0.977	Used for normal estimation

The PC Panel class achieves near-perfect detection across all frames (mean IoU = 0.980), confirming that the panel normal estimation step has robust access to the full surface across all viewpoints. Power socket detection is the strongest among socket classes (mean IoU = 0.892, mean conf. = 0.892), consistent with its large, high-contrast appearance.

USB detection has the lowest mean box IoU (0.520), reflecting the multi-instance ambiguity: YOLO frequently detects 2–5 ports simultaneously, and the bounding boxes of individual ports vary in size depending on viewing angle. The instance-matching stage (Stage 2) resolves this ambiguity by tracking a user-selected reference instance via consensus 3D triangulation.

The PS/2 socket, previously reachable in only one frame at a single confidence score of 0.49, is now detected in 11 of 16 frames with mean IoU = 0.791 and mean confidence 0.747. This reflects an improved YOLO training run with better augmentation coverage for this class and removes the requirement for manual fallback annotation in nominal operation.

5.3 Power Socket Results

Physical dimensions of the IEC C14 face: 30.1×22.6 mm. Pipeline output (run 2): 31.1×25.4 mm, depth prior 6.0 mm. Width overestimates by 3.3%; height overestimates by 12.4% due to SAM2 mask leakage into an adjacent power switch in 2–3 frames. Centre: (0.290, 0.225, 0.526) m in world frame.

5.4 Ethernet Socket Results

Physical RJ45 aperture: $\approx 13.5 \times 9.0$ mm. Pipeline output (run 2): 15.0×13.1 mm, depth prior 1.0 mm. Both dimensions overestimate due to latch notch inclusion in the SAM2 mask and limited clean detections (4 frames after filtering). Centre: (0.289, 0.230, 0.756) m in world frame.

5.5 Final OBB Summary

Table 7: Final OBB extents — all computed sockets

Socket	W (mm)	H (mm)	D (mm)	Source
VGA/DVI	37.8	11.2	6.1	Auto (run 2)
Power (IEC C14)	31.1	25.4	6.0	Auto (run 2)
Ethernet (RJ45)	15.0	13.1	1.0	Auto (run 2)
<i>GT VGA extent: $35.4 \times 11.8 \times 6.1$ mm</i>				

5.6 IoU Comparison: Primary vs Homography Pipeline

The homography-based pipeline achieves reliable boundary propagation for frames with sufficient SIFT matches (≥ 4 inliers). On frames with significant viewpoint change or occlusion, homography estimation fails gracefully (fewer than 4 good matches). Triangulated 3D boundary corners from the homography pipeline serve as an independent estimate of the socket plane; the RANSAC plane fit on these points is consistent with the panel normal estimated from the primary pipeline, providing cross-validation of the depth-axis estimate.

6 Conclusions

We presented two complementary pipelines for world-frame OBB estimation of PC I/O panel sockets from 16 posed RGB images. The key contributions are:

1. **Two-phase multi-instance tracking** resolves the identity ambiguity present in classes with multiple physical instances (e.g. USB ports). A consensus 3D point triangulated from unambiguous frames guides candidate selection in multi-detection frames, requiring only a single user keypress to anchor the reference instance.
2. **Panel normal estimation via SIFT triangulation and RANSAC plane fitting** provides a robust geometric prior for depth-axis disambiguation, yielding a +14.9% absolute gain in 2D IoU over PCA-based baselines. The estimated normal is cached in float64 binary format, eliminating re-estimation overhead on repeat runs.

3. **SAM2-driven voxel carving** replaces classical binary silhouettes with semantically-aware masks, enabling precise segmentation of small metallic objects without task-specific training and generalising to unseen socket classes.
4. **A per-class use_minarearect flag** handles the fundamental instability of `cv2.minAreaRect` for near-square socket point clouds.
5. **A homography-based boundary propagation pipeline** provides an independent, detection-free approach to multi-view socket localisation, suitable when YOLO detections are insufficient.
6. **A manual annotation fallback** with panel-normal integration handles socket classes where automated detection fails, ensuring OBB consistency with the automated pipeline through shared panel normal convention.

The primary pipeline achieves up to **88.5%** 2D polygonal IoU on the VGA/DVI socket and **63.4%** 3D volumetric IoU. Remaining error is attributable to SAM2 mask boundary leakage ($\sim 2\text{--}3$ px) and the coarse 0.5 mm voxel resolution.

Future work. Replacing SIFT with a learned matcher (LightGlue / SuperPoint) would improve normal estimation on low-texture surfaces. Finer voxel resolution (0.25 mm) and mask erosion would reduce boundary leakage. Extending the two-phase instance matching to use epipolar line constraints rather than nearest-neighbour reprojection would make it robust to larger viewpoint changes and sparser unambiguous-frame sets. CoTracker-style learned point tracking could replace the reprojection box filter and provide denser correspondences. For the homography pipeline, iterative homography refinement using all inlier matches (rather than only RANSAC-selected minimal sets) would improve boundary propagation accuracy at large viewpoint changes.

7 Bonus: Novel Ideas Explored

7.1 SAM2 as a Geometric Measurement Tool

A central novelty of this work is repurposing SAM2 — designed for interactive image and video segmentation — as a geometric constraint in 3D reconstruction. Unlike classical space carving, which uses binary silhouette masks derived from background subtraction, we use SAM2’s semantically-aware masks as the carving constraint. This provides two key advantages:

- **Generalisation.** SAM2 generalises to unseen socket classes without retraining. At evaluation time, new socket types receive the same treatment as training classes, as long as YOLO provides a bounding box prompt. Even for missed YOLO detections, a manual box prompt suffices.
- **Richer 3D information.** SAM2’s masks capture non-planar geometry: side walls, mounting nuts, and front face are all segmented consistently across multiple viewpoints. For the VGA socket, SAM2 masks from angled frames include the side walls, providing genuine depth cues that improve voxel carving depth resolution. This would be entirely absent in classical silhouette-based carving.

The result is that voxels representing the full 3D extent of a socket — not just its projected footprint — are captured. For the VGA socket, SAM2 consistently masks the

front face, side walls, and mounting nuts across multiple viewpoints, providing genuine depth information through multi-view consistency.

7.2 Training YOLO with 16 Images: Challenges and the Manual Annotation Fallback

Training a detection model on only 16 images presents fundamental limitations. Each socket class appears in at most 10–12 frames, often at low resolution or partial occlusion. Standard data augmentation (flipping, scaling, colour jitter) partially mitigates overfitting, but detection recall for small or partially occluded sockets remains limited.

The PS/2 socket — clearly visible in only one frame (frame 426, confidence 0.49) — is entirely unreachable by the automated detection pipeline. Rather than accepting this as a hard failure, we designed a **manual annotation fallback** that integrates seamlessly into the automated pipeline:

- If YOLO finds no detections in the user’s reference frame, the pipeline immediately falls back to `cv2.selectROI`: the reference frame is shown and the user draws the bounding box directly by dragging. This manually drawn box is then used as the reference for the instance-matching stage, which proceeds identically to the automated path.
- When coverage after instance matching falls below 15%, the pipeline offers a lower-confidence re-detection pass before invoking the full manual annotation tool.
- When fewer than 3 clean detections survive the reprojection box filter, the pipeline automatically prompts the user to launch the annotation tool `obb_tool_ge_V0.py`.
- The tool provides **multi-view corner triangulation** with epipolar line guidance, Harris corner snap for sub-pixel accuracy, and Huber-robust Levenberg–Marquardt refinement of 3D corner positions.
- Critically, the tool loads the SIFT-estimated panel normal from `panel_normal.npy` and applies it to the OBB computation, so manually annotated OBBs have the **same depth-axis convention** as automated ones, ensuring consistency in the final `answers.json`.

This hybrid approach — automated where data is sufficient, manual where it is not — reflects a practical engineering philosophy for real-world robotics perception tasks where training data is always scarce. It avoids the common failure mode of fully automated pipelines that silently produce incorrect outputs for rare classes.

7.3 Homography as an Independent Boundary Propagation Approach

The homography pipeline explores a fundamentally different philosophy: rather than detecting and segmenting sockets in every frame independently, it propagates a single high-quality manual annotation via homographic transfer. This is motivated by the observation that all sockets lie on a flat planar surface, making the planar homography model exact (up to sensor noise and lens distortion).

Key insights from this approach:

- **Feature tracking and 3D reconstruction are unified.** The TrackManager accumulates SIFT keypoint observations across frames, enabling DLT triangulation from all views simultaneously rather than pairwise.
- **The approach is fully self-contained.** It requires only one annotated keyframe and camera intrinsics/poses, with no detector or segmentation model. This makes it applicable even on hardware without GPU support.
- **Cross-validation of the panel normal.** The RANSAC plane fit on homography-triangulated boundary corners independently estimates the same panel normal as the SIFT-on-panel approach in the primary pipeline, providing cross-validation of the geometric prior.

The primary limitation of the homography approach is its dependence on sufficient SIFT matches: for frames with large viewpoint changes or heavy occlusion, fewer than 4 good matches may be available, causing graceful skip of those frames but reducing the density of triangulated points.

7 References

7 References

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. CVPR*, 2016, pp. 779–788.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] Ultralytics, “YOLOv11,” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] A. Kirillov et al., “Segment anything,” in *Proc. ICCV*, 2023.
- [5] N. Ravi et al., “SAM2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024.
- [6] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *Proc. CVPR*, 2016, pp. 4104–4113.
- [7] K. N. Kutulakos and S. M. Seitz, “A theory of shape by space carving,” *Int. J. Comput. Vis.*, vol. 38, no. 3, pp. 199–218, 2000.
- [8] S. Gottschalk, M. C. Lin, and D. Manocha, “OBBTree: A hierarchical structure for rapid interference detection,” in *Proc. SIGGRAPH*, 1996, pp. 171–180.
- [9] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv preprint arXiv:1801.09847*, 2018.
- [10] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [11] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *Int. J. Comput. Vis.*, vol. 60, no. 2, pp. 91–110, 2004.

- [12] R. Arandjelović and A. Zisserman, “Three things everyone should know to improve object retrieval,” in *Proc. CVPR*, 2012, pp. 2911–2918.
- [13] R. Solovyev, W. Wang, and T. Gabruseva, “Weighted boxes fusion: Ensembling boxes from different object detection models,” *Image Vis. Comput.*, vol. 107, p. 104117, 2021.
- [14] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2003.